

TECHNOLOGY TEAM

Platform Interview Toolkit

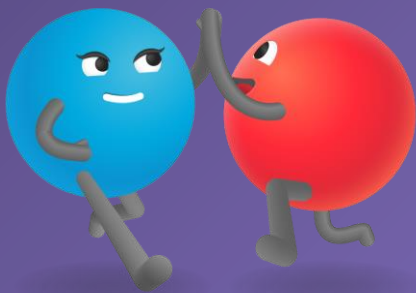


About This Role



At Agoda, we seek Staff & Lead Software Engineers who excel in driving architectural changes & leading development.

They demonstrate strong ownership, design new systems, & enhance team productivity. These engineers provide technical direction, write key components, and understand business impacts.

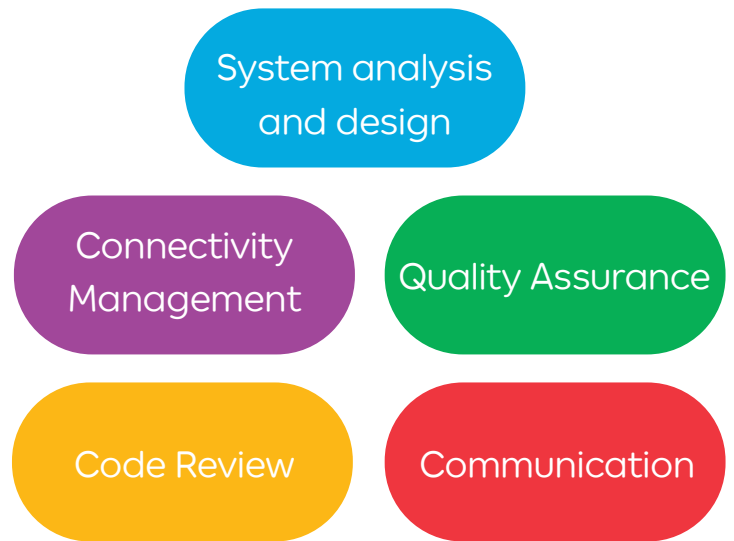


They possess exemplary communication skills, build relationships, & lead cross-team initiatives. Culturally, they show ownership beyond their team, improve technology & people, & serve as role models. They scale expertise, impact multiple teams, & drive strategic initiatives with strong leadership.

At Agoda, our platform interviews are designed to assess your ability to solve real-world engineering challenges through both practical scenarios and conceptual understanding.

The platform interview lasts **60 minutes** and is divided into **2 main parts**: a narrative section and a set of generic questions. The narrative presents a feature or problem, along with diagrams and code snippets, to explore your approach to platform-related topics. The generic questions focus on your ability to apply your knowledge to a variety of use cases, independent of the narrative.

This guide highlights the key areas we evaluate during the platform round, helping you prepare with confidence and clarity.



1

How to Prepare

Practice Platform Scenarios

We strongly encourage preparing by working through **real-world system design** and platform scenarios. Focus on understanding how different components interact, and practice explaining your reasoning clearly.

Here are some recommended topics to focus on:

- Review common system design patterns (e.g., microservices, event-driven architecture)
- Practice designing and documenting APIs (REST, GraphQL, gRPC)
- Work through API error handling and integration strategies
- Explore different testing strategies (unit, integration, end-to-end, non-functional)
- Review code snippets for readability, maintainability, and best practices
- Engage in mock interviews or technical discussions with peers

Solving practical problems and discussing your approach will help you build confidence and improve your ability to communicate complex ideas.

2 Interview Attributes

System Analysis & Design

You'll be presented with a system design scenario, often with intentional gaps or faults. We want to see how you analyze requirements, identify issues, and propose improvements.

- Practice breaking down system requirements and identifying potential bottlenecks or missing pieces.
- Review basic system design concepts: scalability, integration, fault tolerance, and trade-offs (performance, cost, security, etc.).
- Be ready to explain your design choices and consider alternatives.

Pro Tip: Think out loud! Walk the interviewer through your thought process and highlight the pros and cons of your decisions.

Connectivity Management

You'll discuss API design and connectivity frameworks. Expect questions about protocol choices, error handling, and integration strategies.

- Review REST, GraphQL, gRPC, and when to use each.
- Understand API error handling, authentication/authorization, and synchronous vs. asynchronous communication.
- Practice designing API contracts and explaining your reasoning.

Pro Tip: Be specific about why you choose a particular approach and mention best practices you follow in real projects.

Quality Assurance

We'll ask about your approach to testing and ensuring system quality, both at the design and code levels.

- Brush up on testing strategies: unit, integration, end-to-end, and non-functional tests.
- Know the purpose and trade-offs of different test types (e.g., test pyramid).
- Be ready to suggest ways to improve test coverage and reliability.

Pro Tip: Start with "why" when discussing tests—explain the value each test brings to the system.

Code Review



You'll review code snippets, spot issues, and suggest improvements. We're looking for your ability to apply clean code principles and provide constructive feedback.

- Practice reviewing code for readability, maintainability, and correctness.
- Familiarize yourself with common code smells, SOLID principles, and basic design patterns.
- Be ready to suggest fixes and explain your reasoning.

Pro Tip: Frame your feedback positively and offer alternatives, not just criticisms

Communication is Key

We value clear, effective communication. You'll be assessed on how well you explain your ideas and engage in technical discussions.

- Practice explaining complex concepts simply and logically.
- Engage in mock interviews or technical discussions with peers.
- Be open to questions and clarify your reasoning as you go.

Pro Tip: Adapt your explanations to your audience—show that you can communicate with both technical and non-technical stakeholders.

3

Resources to Help You Prepare

Here are some useful resources to guide your preparation:

- [System Design Primer](#) – Comprehensive guide to system design concepts and interview questions
- [API Design Guide \(Swagger\)](#) – Best practices for designing robust APIs
- [GeeksforGeeks System Design](#) – Tutorials and examples on system design topics
- [Refactoring Guru](#) – Clean code principles and design patterns explained
- [Testing Strategies \(Martin Fowler\)](#) – Insights into test strategies and the test pyramid
- [Awesome Code Review](#) – Curated resources for effective code reviews
- [Pramp](#) & [Interviewing.io](#) – Free mock interview sessions for system design and technical interviews

Note: These are external resources and are not affiliated with Agoda.